

Exception Package

The `exception` package handles error scenarios within the application by defining custom runtime exceptions and JAX-RS `ExceptionHandler` classes to return standardized JSON error responses.

Files

`BadRequestException.java` & `ResourceNotFoundException.java`

These are custom runtime exceptions used to signal client-side errors (400) and missing resources (404).

```
public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) {
        super(message);
    }
}
```

Explanation Both classes extend `RuntimeException`, allowing them to be thrown anywhere in the service logic without being explicitly declared in method signatures. They store a descriptive error message passed to the super constructor.

`BadRequestExceptionHandler.java` & `ResourceNotFoundExceptionHandler.java`

These classes intercept the custom exceptions and transform them into HTTP `Response` objects.

```
@Provider
public class BadRequestExceptionHandler implements ExceptionHandler<BadRequestException> {
    @Override
    public Response toResponse(BadRequestException exception) {
        ErrorResponse error = new ErrorResponse("400", "Bad Request", exception.getMessage());
        return Response.status(Response.Status.BAD_REQUEST)
            .entity(error)
            .type(MediaType.APPLICATION_JSON)
            .build();
    }
}
```

Line-by-line Explanation

- **@Provider:** Marks the class as a JAX-RS component that can be discovered during package scanning.

- **implements `ExceptionHandler<T>`**: Indicates this class is responsible for mapping a specific exception type.
- **`toResponse(T exception)`**: The method where the exception is caught and the HTTP response is constructed.
- **`new ErrorResponse(...)`**: Creates a structured error object containing a code, reason, and specific message.
- **`Response.status(...).entity(error)...build()`**: Builds a JAX-RS response with the appropriate status code and JSON body.

Analysis

- **Annotations:**
 - **`@Provider`**: Essential for JAX-RS to register the mapper automatically.
- **Architectural Decision:**
 - **Exception Mapping**: This pattern (Global Exception Handling) decouples error response logic from the business logic. Instead of using try-catch blocks in every controller method, we simply throw an exception and let the mapper handle the rest.
 - **ErrorResponse Model**: Using a dedicated model (**`ErrorResponse`**) ensures that all errors returned by the API have a consistent structure, which is crucial for client-side integration.